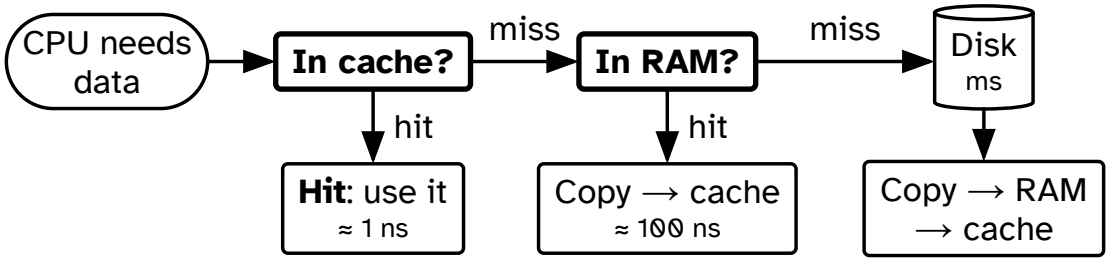
Foundations

Computer architecture

	What	Size	Speed	Persists?
Cache	On the CPU die, SRAM. L1 < L2 < L3 in size, L1 is the fastest	KB–10s MB	ns	No
RAM	Open apps and variables (DRAM)	1–128 GB	µs / MB	No
SSD	Flash, no moving parts	TB	ms / MB	Yes
HDD	Spinning platter + head, wears out	TB	slower	Yes

- Speed: cache > RAM > SSD > HDD. Size and cheapness go in the reverse order.
- **RAM ↔ disk never talk directly.** The CPU sits between them.
- **CPU loop:** fetch → decode → execute, and each instruction takes under 1 ns.
- **Read path:** cache → RAM → disk. A hit is served fast; a miss goes down a level.
- CPU caches are managed by **hardware**. The OS manages the **page cache** (disk data kept in RAM).
- **Cache** = a smaller, faster copy of slower data (CPU, browser caching on disk, ...).
- **Moore:** transistors ×2 every ≈ 2 years: $N = N_0 \cdot 2^{t/2}$. Exponential, now plateauing.



Read path: check the fastest place first

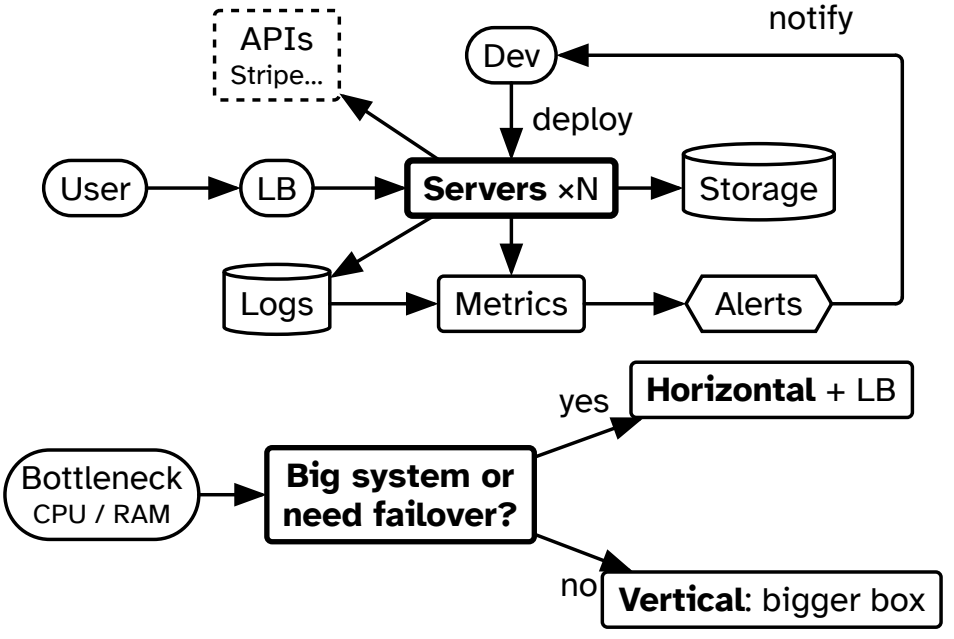
Application architecture

- **Dev:** code → build & deploy → **server** → external **storage** (DB / cloud over the network).
- **User:** browser (client) → **load balancer** → servers → HTML / CSS / JS back.
- **Bottleneck** (CPU or RAM) → **scale**:

	Vertical (up)	Horizontal (out)
How	Bigger machine	More machines + LB
Pros	Easy	No ceiling, survives failure, cheap hardware
Cons	Hard ceiling, single point of failure	Engineering effort: coordination, spreading load

- External services via **APIs** (e.g. Stripe).

- **Logs** = what happened (store them externally). **Metrics** = CPU / RAM / network numbers. **Alerts** = a push when a metric misses its target (e.g. success rate < 95%).



Request flow, observability loop and the scaling decision

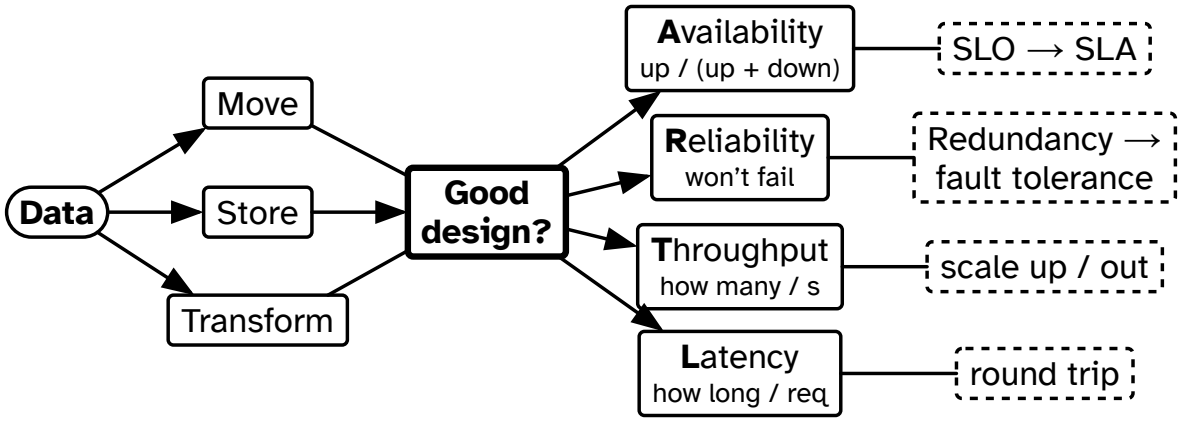
Design requirements

- **Every system: Move · Store · Transform** data.
- Bad code is cheap to fix. Bad design is costly (DB migration + rewrite).
- **Judge a design by "A Real Tough Load":** Availability · Reliability · Throughput · Latency.

$$\text{availability} = \frac{\text{uptime}}{\text{uptime} + \text{downtime}}$$

99%	99.9%	99.99%	99.999%
3.65 days/yr	8.76 h/yr	52.6 min/yr	5.26 min/yr

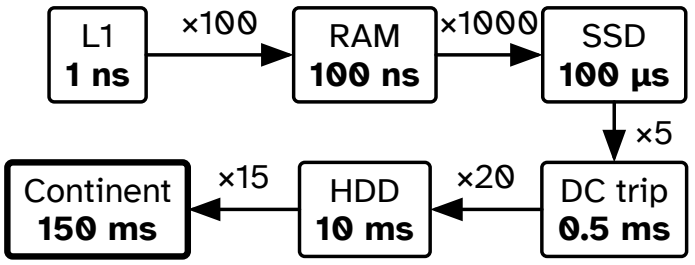
- Each extra 9 means 10× less downtime. 23/24 h ≈ 96%, which is poor.
- **SLA** = the promise to customers (AWS: 99.99%/month, or credits back). **SLO** = the internal target that meets the SLA.
- **Redundancy → fault tolerance → reliability.** Active-passive: the backup is idle. Active-active: both serve traffic.
- Under a DDoS, redundancy alone fails, because the backup gets flooded too.
- **Throughput** = how many per second: req/s, QPS (user-facing), bytes/s (pipelines). It is always ≤ the **bandwidth**.
- **Latency** = how long one request takes (round trip). Highway analogy: throughput is cars per hour, latency is one car's trip time.



What a system does, and how to judge it

Numbers to know (approximate)

Operation	Time
L1 cache read	≈ 1 ns
RAM read	≈ 100 ns
SSD random read	≈ 100 µs
HDD seek	≈ 10 ms
Round trip inside a datacentre	≈ 0.5 ms
Round trip across continents	≈ 150 ms



The latency ladder: from L1 to a trip across continents is ≈ 150 million× slower